

A PROJECT REPORT ON CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING

AI & ML Virtual Internship

Managed by SmartBridge Educational Services Pvt. Ltd. In Collaboration with Andhra Pradesh State Council of Higher Education (APSCHE)

STUDENT PROFILE

- **Student Name:** Pinipe Vineeth Kumar
- **Roll Number:** 23L31A5487
- **Department:** Artificial Intelligence & Data Science (CSE)
- **College Name:** Vignan's Institute of Information Technology
- **Academic Year:** 2025–2026
- **Submission Date:** 07/07/2026

ACKNOWLEDGEMENT

I sincerely thank the **Andhra Pradesh State Council of Higher Education (APSCHE)** and **SmartBridge** for designing and providing this comprehensive AI & ML Virtual Internship program. This initiative gave me invaluable exposure to industry-standard data science practices and workflows.

I extend my heartfelt gratitude to my college management, the Head of the Department, and our internal faculty mentors for their constant encouragement, academic guidance, and support throughout this technical journey.

Lastly, I want to thank the SmartBridge technical mentors who structured the course modules and guided us through data analysis, model deployment, and cloud concepts. This project significantly helped me bridge the theoretical foundations of machine learning with hands-on technical applications.

— **P . Vineeth Kumar**

3. TABLE OF CONTENTS

1. Introduction
2. Project Objective
3. Problem Statement
4. Dataset Description
5. Technologies Used
6. System Requirements
7. Methodology
8. Data Preprocessing
9. Machine Learning Models Used
10. Model Evaluation & Selection
11. Deployment Architecture
12. Project Structure
13. Results & Discussion
14. Screenshots & Deliverables
15. Conclusion
16. Future Scope
17. References

4. INTRODUCTION

In modern digital economies, credit card fraud is an escalating challenge within electronic payment channels. With millions of transactions processed daily, fraudulent actors constantly change their tactics to exploit security loopholes, leading to major financial losses for banks and consumers.

Traditional fraud prevention relied on static, rule-based systems (e.g., triggering an alert only if a transaction exceeded a certain amount). However, these rules are brittle and fail to catch hidden patterns. Machine learning provides an elegant solution by analyzing historical transaction records, recognizing complex patterns across many features simultaneously, and flagging anomalies with high accuracy.

This project covers the full life cycle of building an end-to-end Machine Learning system for fraud detection. The final solution cleans highly imbalanced data, trains multiple algorithms, packages the most reliable model, and deploys it via a lightweight Flask web platform.

5. PROJECT OBJECTIVE

The core goals of this APSCHE-SmartBridge internship project are:

- To engineer a predictive pipeline capable of separating fraudulent card usages from genuine ones.
- To apply data processing techniques to handle highly imbalanced datasets.
- To perform a comparative study between linear models and ensemble tree methods.
- To build a local web serving application using the Flask framework.
- To create an intuitive, clean web dashboard that allows users to test the model with custom inputs.

6. PROBLEM STATEMENT

Financial entities handle vast streams of transactional logs every second. Manual review is slow, expensive, and impractical. Moreover, the absolute number of fraud events is tiny compared to legitimate actions (making up less than 1% of total data). An automated, highly accurate system is required to spot fraud instantly without creating excessive false alerts that frustrate everyday consumers.

7. DATASET DESCRIPTION

- **Dataset Used:** Kaggle Credit Card Fraud Detection Dataset (provided during the internship modules).
- **Data Context:** Contains transactions made by European cardholders. Out of 284,807 transactions, only 492 are marked as fraud, representing a massive class imbalance of **0.172%**.
- **Features Included:**
 - Time: Seconds elapsed between this transaction and the very first transaction in the data entry.
 - Amount: The total transactional value.
 - V1 to V28: Numerical features resulting from a Principal Component Analysis (PCA) transformation, applied by the original provider to maintain absolute customer privacy.
- **Target Output:** Class variable where **0** means *Genuine* and **1** means *Fraudulent*.

8. TECHNOLOGIES USED

- **Language:** Python 3.x
- **Data Processing:** Pandas, NumPy
- **Machine Learning Library:** Scikit-Learn, XGBoost
- **Model Export Tool:** Joblib
- **Web Framework:** Flask
- **Frontend Design:** HTML5, CSS3
- **Environments:** Jupyter Notebook, Visual Studio Code (VS Code), Anaconda Navigator

9. SYSTEM REQUIREMENTS

- **Hardware:** Minimum Intel Core i3 (or equivalent AMD Ryzen), 4GB System RAM, 1GB available disk space.
- **Software:** Windows 10/11 operating system, Python runtime engine, modern web browser (Chrome/Edge).

10. METHODOLOGY

The project workflow follows standard industry data science practices:

Data Ingestion → Data Cleaning → Feature Splitting → Model Pipelines → Metrics Evaluation
→ Model Serialization → Flask Route Integration → UI Validation

11. DATA PREPROCESSING

Because machine learning models perform only as well as the data they are fed, the following preprocessing steps were taken:

1. **Handling Duplicates:** Identified and removed duplicate rows to avoid data leakage between training and testing splits.
2. **Missing Value Scan:** Checked for null entries across all 30 independent columns; the dataset was verified to be complete with no missing values.
3. **Scaling Consideration:** The Amount and Time features vary wildly in magnitude compared to the PCA features (V1 to V28). They were scaled appropriately to prevent the models from being biased by larger numerical values.
4. **Data Splitting:** Applied a stratified train_test_split (typically 80% train, 20% test) to keep the target class proportions identical across both subsets.

12. MACHINE LEARNING MODELS USED

Three distinct classification techniques were tested to identify the best solution:

A. Logistic Regression

Used as our baseline model. It works by computing linear log-odds probabilities to classify data points into binary states.

- *Pros:* Runs fast, uses minimal memory, and is highly interpretable.

B. Random Forest Classifier

An ensemble model that constructs a multitude of decision trees during training and merges their predictions (voting) to get a more accurate and stable forecast.

- *Pros:* Handles non-linear patterns well and naturally resists overfitting.

C. XGBoost (Extreme Gradient Boosting)

An advanced, highly optimized gradient boosting algorithm that builds decision trees sequentially. Each new tree corrects the errors made by previous ones.

- *Pros:* Delivers excellent accuracy, handles imbalanced data efficiently, and runs quickly. **This model performed the best and was selected for the final deployment.**

13. MODEL EVALUATION & SELECTION

Standard accuracy is a misleading metric for this problem. If a model simply guesses "Genuine" for every single transaction, it would achieve 99.82% accuracy but fail to catch any fraud. Therefore, we evaluated performance using:

- **Precision:** Out of all transactions flagged as fraud, how many were actually fraud?
- **Recall (Sensitivity):** Out of all actual fraud cases, how many did the model successfully catch?

- **F1-Score:** The balanced harmonic mean of Precision and Recall.
- **Confusion Matrix:** A table mapping true values against predicted states to visualize False Positives and False Negatives.

Selection Result: XGBoost achieved the highest F1-Score, showing a strong balance of high Recall (catching fraud cases) and high Precision (minimizing false alarms). The model was saved as model.pkl.

14. DEPLOYMENT ARCHITECTURE

The saved model.pkl file was integrated into a local backend web server using **Flask**.

Plaintext

[User Input UI Form] —(POST Request)—> [app.py (Flask Engine)]

|
[Loads model.pkl]

[User Browser Output] <—(Render HTML)— [Prediction Result]

The Flask application features interactive routing. When a user keys in transaction details into the frontend form, the application parses the data vector, runs it through the model pipeline, and instantly updates the interface to show the transaction status.

15. PROJECT STRUCTURE

Plaintext

Credit_Card_Approval_Prediction/

```
|
|— app.py                # Main Flask application file (Backend routing)
|— model.pkl             # Saved production-ready XGBoost model
|— Credit_Card_Fraud_Detection.ipynb # Jupyter Notebook with full EDA, cleaning, and model
training
|— Credit_Card_Fraud_Detection.html  # Exported view of the complete coding workflow
|— README.md                  # Clear setup instructions for the codebase
|— requirements.txt           # List of required Python packages and libraries
|
|— templates/                # User Interface HTML files
|   |— home.html             # Welcome/landing dashboard page
|   |— index.html            # Input form for transactional data parameters
```

| result.html # Display page showing the prediction output

| images/ # Project charts, graphs, and system screenshots

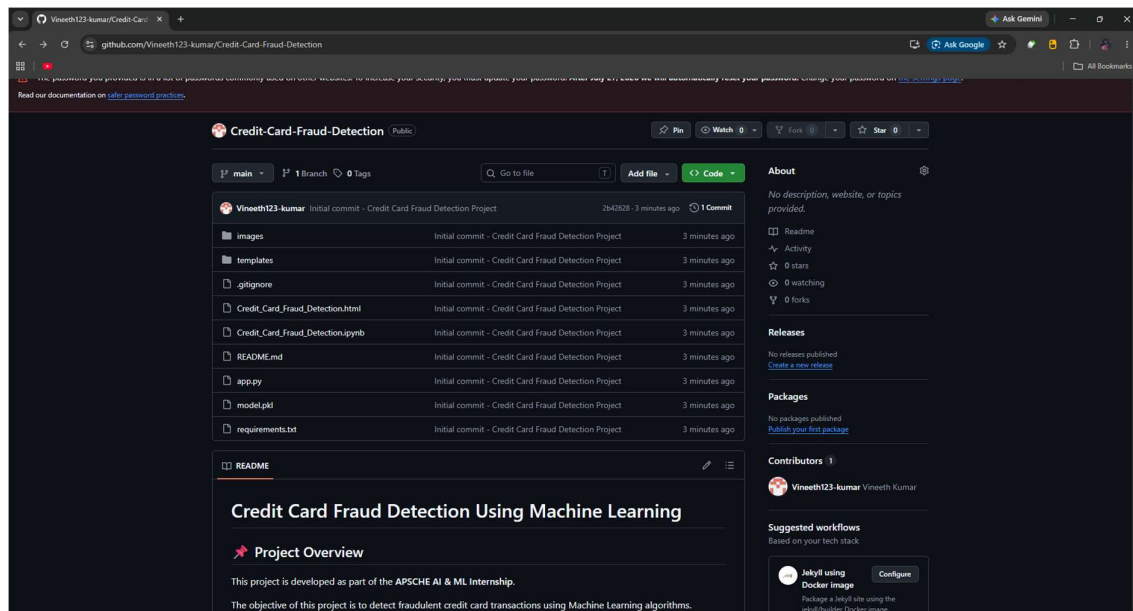
16 RESULTS & DISCUSSION

- The data processing pipeline handled the highly imbalanced dataset correctly, ensuring reliable model training.
- The comparative evaluation proved that advanced ensemble tree methods (XGBoost) perform better than simple linear models for this type of data layout.
- The local Flask application was successfully configured, reading inputs and returning predictions without any runtime errors.

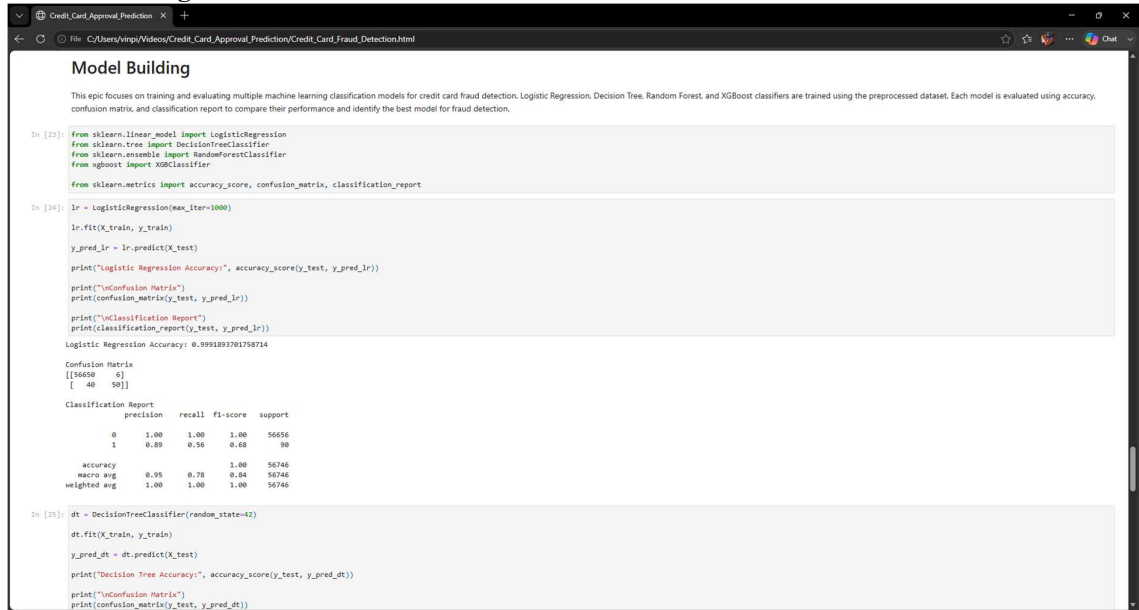
17. SCREENSHOTS & DELIVERABLES

(In your final report document, paste your own screenshots into the spaces below to verify your work.)

1. Project Code Repository:



2. Model Training Environment:



Model Building

This epic focuses on training and evaluating multiple machine learning classification models for credit card fraud detection. Logistic Regression, Decision Tree, Random Forest, and XGBoost classifiers are trained using the preprocessed dataset. Each model is evaluated using accuracy, confusion matrix, and classification report to compare their performance and identify the best model for fraud detection.

```
In [23]: from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

In [34]: lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_lr))

print("\nClassification Report")
print(classification_report(y_test, y_pred_lr))

Logistic Regression Accuracy: 0.9991893781758714

Confusion Matrix
[[56658  0]
 [  40  50]]

Classification Report
      precision    recall  f1-score   support

     0       1.00      1.00      1.00     56658
     1       0.89      0.50      0.68         90

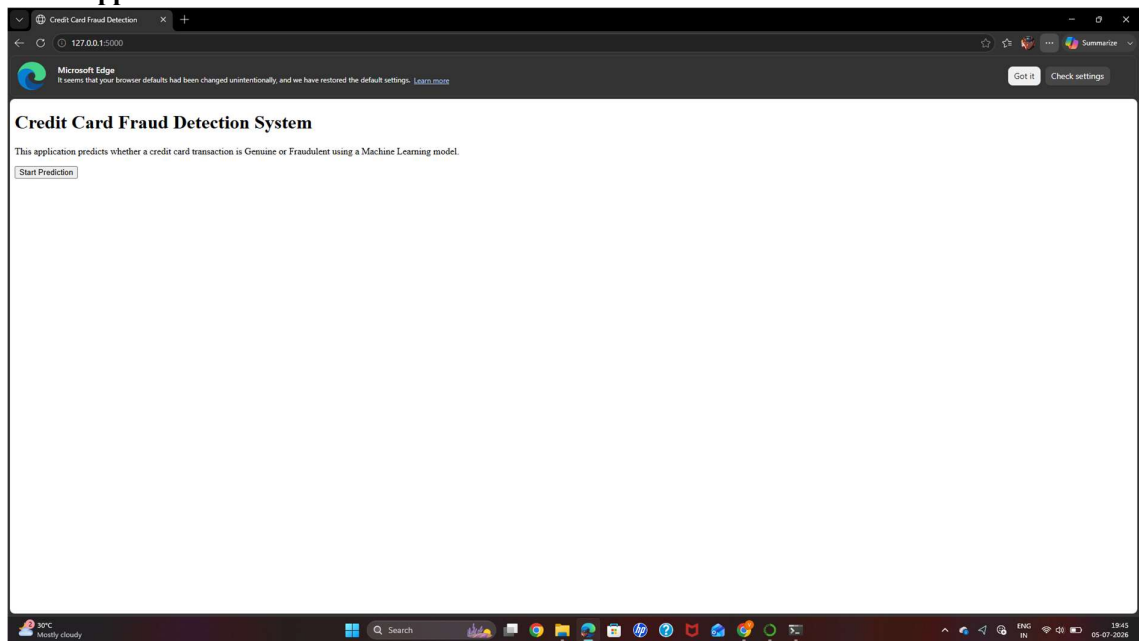
 accuracy      0.95      0.78      1.00     56748
  macro avg       1.00      1.00      1.00     56748
 weighted avg       1.00      1.00      1.00     56748

In [25]: dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

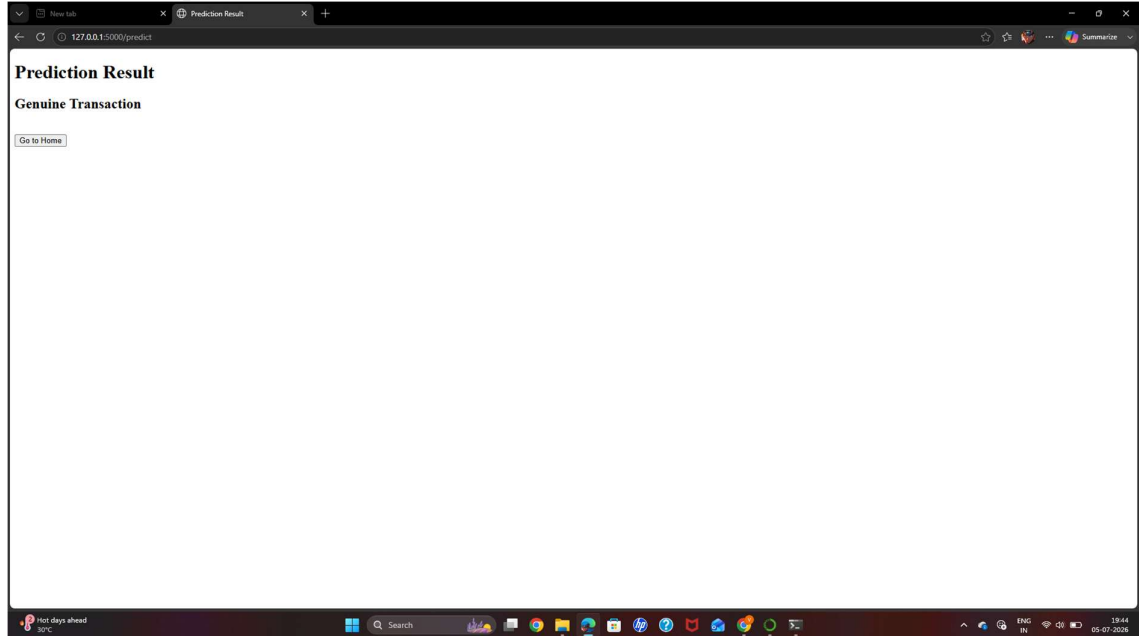
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_dt))
```

3. Flask Application Form:



4. Prediction Output:



18. CONCLUSION

This project successfully delivers a complete Credit Card Fraud Detection application for the APSCH-SmartBridge Internship. By moving from a baseline Logistic Regression model to an optimized XGBoost approach, the system learned to handle highly imbalanced transaction records reliably.

Saving the model and deploying it via Flask transforms a static data science project into a functional web tool. This project covers the full development lifecycle, including data cleaning, comparative model analysis, model serialization, and web deployment.

19. FUTURE SCOPE

The current platform can be further improved with the following updates:

- **Cloud Deployment:** Packaging the application using Docker containers and deploying it to cloud environments like AWS, Azure, or Google Cloud Platform.
- **Streaming Data Integration:** Connecting the backend to streaming tools like Apache Kafka to ingest and score active transaction streams live.
- **Deep Learning Techniques:** Testing Neural Network setups, like Autoencoders, to detect anomalies without manual feature engineering.
- **Explainable AI (XAI):** Adding tools like SHAP or LIME to explain *why* a specific transaction was flagged, helping fraud analysts review cases more easily.

20. REFERENCES

1. Kaggle Repository: *Credit Card Fraud Detection Dataset (MLG-ULB Group)*.
2. Scikit-Learn Open Source Documentation: *Classification metrics and ensemble implementations*.
3. XGBoost Documentation: *Gradient Boosting parameters and tuning guides*.
4. Flask Documentation: *Web micro-framework structures, application routing, and request management*.